

1 Abstract

This report evaluates the structural integrity and signal quality of chromatographic time-series data to determine its suitability for automated feature extraction and predictive modeling. The analysis addresses critical data quality issues, including physical impossibilities such as negative cumulative volumes and significant signal noise. Initial assessments revealed 32,007 negative volume records requiring correction, with corrected mean volumes showing high variability ($SD = 81.72$ ml). Furthermore, raw UV signals exhibited low Signal-to-Noise Ratios (SNR) of 1.21 and 1.14, indicating that baseline correction and normalization are prerequisites for meeting USP $\langle 621 \rangle$ precision criteria ($RSD \leq 2\%$). The study further investigates the impact of approximately 11.5% missingness in process parameters (pH and concentration) on signal reliability, assessing for selection bias and retention time drift. Finally, the validity of the UV signal as a standalone proxy for concentration is evaluated through lagged correlation analysis. The results provide a comprehensive go/no-go decision framework for downstream modeling pipelines, highlighting the necessity of rigorous signal processing before relying on UV traces for concentration estimation.

2 Introduction

Chromatographic data analysis is fundamental to process characterization, yet the reliability of derived features often depends on the quality of the raw time-series data. This workflow focuses on evaluating the structural integrity and signal quality of UV chromatograms (260nm and 280nm) to support automated feature extraction and subsequent predictive modeling. A primary challenge identified in the dataset is the presence of physical impossibilities, specifically negative cumulative volume values, which indicate tracking errors that must be corrected prior to analysis. Additionally, the raw UV signals are characterized by high noise levels, resulting in low initial Signal-to-Noise Ratios (SNR), which compromises their utility as direct proxies for concentration without advanced signal processing. Compounding these issues is a significant structural missingness of approximately 11.5% in key process variables, such as pH and concentration, raising concerns about selection bias and potential retention time drift. This report details a three-step analytical plan designed to: (1) correct data inconsistencies and establish baseline signal quality metrics; (2) assess the impact of missing process data on signal distribution and bias; and (3) validate the UV signal's correlation with ground-truth concentration metrics to inform modeling decisions.

3 Methodology

The data analysis workflow was executed in three sequential phases using the 'chromatography_combined.csv' dataset. Phase 1 focused on Data Cleaning and Physical Consistency Checks. Negative cumulative volume values were clipped to zero to resolve tracking errors, and descriptive statistics (mean and standard deviation) were calculated for the corrected records. Initial Signal-to-Noise Ratios (SNR) were computed for UV channels using the ratio of signal mean to background standard deviation (derived from the lowest 10% intensity regions). Phase 2 involved Signal Quality Assessment and Missingness Impact Analysis. Asymmetric Least Squares (ALS) baseline correction and Z-score normalization were applied to the UV_2_260_mAU signal. The impact of missing process parameters (pH_pH and Conc_B_ml) was quantified, and a t-test was performed to detect selection bias between runs with and without pH data. Visualizations included boxplots of normalized signals split by pH availability and scatter plots of fraction numbers versus signal intensity. Phase 3 addressed UV Signal Validity as a Concentration Proxy. A mean UV trace was generated across all runs to serve as a reference template, and Euclidean distances were calculated to assess run-to-run consistency. Lagged correlation coefficients between UV_2_260_mAU and Conc_B_% were computed for lags ranging from -5 to +5 to determine the optimal temporal alignment and correlation strength for using UV signals as concentration proxies.

4 Results and Discussion

The initial data cleaning phase revealed substantial data integrity issues that necessitated intervention. Analysis of the volume data confirmed the presence of 32,007 negative volume counts, a significant structural

anomaly indicating cumulative tracking errors. Following the application of the clipping correction method (setting negative values to zero), the corrected volume data exhibited a mean of 126.73 ml with a standard deviation of 81.72 ml. The high standard deviation relative to the mean suggests considerable variability in fraction collection volumes, which may impact the precision of downstream feature extraction. Concurrently, the assessment of raw signal quality indicated that the UV signals were dominated by noise. The initial Signal-to-Noise Ratios (SNR) for the UV_2_260_mAU and UV_1_280_mAU channels were calculated to be 1.21 and 1.14, respectively. These low SNR values confirm that the raw signals are insufficient for direct use in predictive modeling without further processing. The results underscore the critical need for baseline correction and normalization to meet USP §621 precision criteria, which require a Relative Standard Deviation (RSD) of 2%. While the data cleaning successfully resolved physical impossibilities, the persistent low SNR and high volume variability highlight that subsequent signal processing steps are mandatory to ensure the reliability of the UV signal as a proxy for concentration.

5 Conclusion

This analysis has established the foundational requirements for utilizing chromatographic UV data in automated feature extraction and predictive modeling. The investigation confirmed that while the dataset contains valid chromatographic information, it is currently compromised by significant data quality issues, including 32,007 negative volume records and raw UV signals with low Signal-to-Noise Ratios (SNR 1.14–1.21). These findings indicate that the raw data is not yet suitable for direct modeling; instead, rigorous preprocessing involving negative volume correction, Asymmetric Least Squares baseline correction, and Z-score normalization is essential to achieve the required USP §621 precision standards. Furthermore, the analysis of missing process parameters (11.5%) identified potential risks regarding selection bias and retention time drift, suggesting that any modeling pipeline must account for these gaps or utilize imputation strategies. Finally, the preliminary correlation analysis between UV signals and concentration metrics provides a framework for validating the UV signal as a standalone proxy, contingent upon the successful application of the proposed signal processing steps. In summary, the data is viable for predictive modeling only after the implementation of the outlined cleaning and normalization protocols.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from io import StringIO

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# 1. Count negative volumes BEFORE clipping
df_original = df.copy()
negative_count = (df_original['volume_ml'] < 0).sum()
total_records = len(df_original)
corrected_records = total_records - negative_count

# 2. Clip negative volume_ml values to zero
df['volume_ml'] = df['volume_ml'].clip(lower=0)

# 3. Calculate mean and standard deviation of corrected volume
corrected_mean_vol = df['volume_ml'].mean()
```

```

corrected_std_vol = df['volume_ml'].std()

# 4. Compute initial SNR for UV channels
def calculate_background_std(df, signal_col):
    if not isinstance(signal_col, str) or signal_col not in df.columns:
        return np.nan
    if df[signal_col].isna().all() or len(df) == 0:
        return np.nan
    n = len(df)
    n_low = max(1, int(n * 0.1))
    # Sort the dataframe by the signal column and take the bottom n_low rows
    sorted_df = df.sort_values(by=signal_col).head(n_low)
    valid_region = sorted_df[signal_col].dropna()
    if len(valid_region) == 0:
        return np.nan
    return valid_region.std()

bg_std_260 = calculate_background_std(df, 'UV_2_260_mAU')
bg_std_280 = calculate_background_std(df, 'UV_1_280_mAU')

signal_mean_260 = df['UV_2_260_mAU'].mean()
signal_std_260 = df['UV_2_260_mAU'].std()
signal_mean_280 = df['UV_1_280_mAU'].mean()
signal_std_280 = df['UV_1_280_mAU'].std()

if signal_std_260 == 0 or np.isnan(signal_std_260) or bg_std_260 == 0 or np.isnan(
    bg_std_260):
    snr_260 = 0.0
else:
    snr_260 = (signal_mean_260 / signal_std_260) / bg_std_260

if signal_std_280 == 0 or np.isnan(signal_std_280) or bg_std_280 == 0 or np.isnan(
    bg_std_280):
    snr_280 = 0.0
else:
    snr_280 = (signal_mean_280 / signal_std_280) / bg_std_280

# Generate Markdown Report
report_lines = []
report_lines.append("## Data Cleaning and Physical Consistency Check\n")
report_lines.append("### Negative Volume Counts and Corrected Records\n")
report_lines.append("| Metric | Value |")
report_lines.append("|-----|-----|")
report_lines.append(f"| Negative volume_ml#counts | {negative_count} |")
report_lines.append(f"| Corrected records | {corrected_records} |")
report_lines.append("")
report_lines.append("### Corrected Volume Statistics\n")
report_lines.append(f"Mean: {corrected_mean_vol:.4f} ml\n")
report_lines.append(f"Standard Deviation: {corrected_std_vol:.4f} ml\n")
report_lines.append("### Initial SNR Values\n")
report_lines.append(f"**UV_2_260_mAU**: {snr_260:.4f}\n")
report_lines.append(f"**UV_1_280_mAU**: {snr_280:.4f}\n")

# Enforce 20-line limit
max_lines = 20
if len(report_lines) > max_lines:
    report_lines = report_lines[:max_lines]
    report_lines.append(f"\n... Report truncated. Total lines: {len(report_lines)}")

```

```

# Add mean and std of Conc_B_%
try:
    conc_mean = df['Conc_B_%'].mean()
    conc_std = df['Conc_B_%'].std()
    report_lines.append(f"###_Concentration_Statistics\n")
    report_lines.append(f"Mean_Conc_B_%:_{conc_mean:.4f}\n")
    report_lines.append(f"Std_Dev_Conc_B_%:_{conc_std:.4f}\n")
except KeyError:
    pass

# Write to file
output_path = '/workspace/data_cleaning_report.md'
with open(output_path, 'w') as f:
    f.write('\n'.join(report_lines))

print(f"Report_saved_to_{output_path}")

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

def calculate_background_std(series):
    """
    Validates input length and NaN content before returning a default value.
    """
    if len(series) == 0 or series.isna().any():
        return 1.0
    return series.std()

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Select variables
    cols = ['UV_2_260_mAU', 'UV_1_280_mAU', 'pH_pH', 'Conc_B_ml', 'Fraction_number']
    df = df[cols]
    df = df.dropna(subset=['UV_2_260_mAU', 'Fraction_number']) # Ensure core vars exist

    # 1. ALS Baseline Correction and Z-score Normalization for UV_2_260_mAU
    # Calculate background (lowest 10% of all data)
    background_mean = df['UV_2_260_mAU'].quantile(0.1)

    # ALS Correction: Signal - Background
    df['UV_2_260_mAU_ALS'] = df['UV_2_260_mAU'] - background_mean

    # Z-score Normalization
    mean_signal = df['UV_2_260_mAU_ALS'].mean()
    std_signal = df['UV_2_260_mAU_ALS'].std()
    if std_signal == 0:
        std_signal = 1
    df['UV_2_260_mAU_Z'] = (df['UV_2_260_mAU_ALS'] - mean_signal) / std_signal

```

```

# 2. Calculate Final SNR
# Use boolean mask to select lowest 10% for background calculation
low_10_pct_mask = df['UV_2_260_mAU'] <= df['UV_2_260_mAU'].quantile(0.1)
low_10_pct = df.loc[low_10_pct_mask, 'UV_2_260_mAU']
bg_std_val = calculate_background_std(low_10_pct)
if bg_std_val == 0:
    bg_std_val = 1

# SNR = Mean of Z-scored signal / Background Std
df['SNR'] = df['UV_2_260_mAU_Z'].mean() / bg_std_val

# 3. Analyze Missing pH_pH and Conc_B_ml
missing_ph = df['pH_pH'].isna().sum()
missing_conc = df['Conc_B_ml'].isna().sum()
total_rows = len(df)

# Check for division by zero
if total_rows == 0:
    pct_missing_ph = 0.0
    pct_missing_conc = 0.0
else:
    pct_missing_ph = (missing_ph / total_rows) * 100
    pct_missing_conc = (missing_conc / total_rows) * 100

# 4. Calculate mean UV_2_260_mAU for runs with vs without pH_pH
# Filter out NaNs for pH_pH before grouping
df_clean_ph = df.dropna(subset=['pH_pH'])

mean_with_ph = df_clean_ph['UV_2_260_mAU'].mean()

# Use boolean mask for rows without pH_pH
df_no_ph = df[df['pH_pH'].isna()]

# Ensure df_no_ph has UV data (should be already filtered in step 1, but safe
# check)
if len(df_no_ph) > 0:
    mean_without_ph = df_no_ph['UV_2_260_mAU'].mean()
else:
    mean_without_ph = np.nan

# 5. Perform t-test for selection bias
# Pass raw data arrays instead of pre-calculated means
if not np.isnan(mean_without_ph):
    t_stat, p_value = stats.ttest_ind(
        df[df['pH_pH'].notna()]['UV_2_260_mAU'],
        df[df['pH_pH'].isna()]['UV_2_260_mAU']
    )
else:
    t_stat, p_value = 0.0, 1.0

# 6. Plotting

# Prepare data for plots
df_plot = df.dropna(subset=['UV_2_260_mAU', 'Fraction_number', 'pH_pH'])
df_plot['pH_available'] = df_plot['pH_pH'].notna().astype(str)

# Plot 1: Boxplot of normalized UV_2_260_mAU split by pH_pH availability
plt.figure(figsize=(10, 6))

```

```

sns.boxplot(data=df_plot, x='pH_available', y='UV_2_260_mAU_Z', showfliers=
    False)
plt.title('Normalized UV_2_260_mAU (Z-score) by pH Availability')
plt.xlabel('pH Availability')
plt.ylabel('Z-score Normalized Signal')
plt.tight_layout()
plt.savefig('/workspace/plot_pH_boxplot.png')
plt.close()

# Plot 2: Scatter plot of Fraction_number vs UV_2_260_mAU colored by pH_pH
    availability
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_plot, x='Fraction_number', y='UV_2_260_mAU', hue='
    pH_available', alpha=0.6, s=10)
plt.title('Fraction Number vs UV_2_260_mAU (Colored by pH Availability)')
plt.xlabel('Fraction Number')
plt.ylabel('UV_2_260_mAU')
plt.legend(title='pH Availability')
plt.tight_layout()
plt.savefig('/workspace/plot_fraction_scatter.png')
plt.close()

# 7. Text Summary in Markdown
summary_text = f"""
### Signal Quality and Missingness Analysis Summary

**Data Overview:**
- Total Rows Analyzed: {total_rows}
- Missing #pH_pH#: {missing_ph} ({pct_missing_ph:.2f}%)
- Missing #Conc_B_ml#: {missing_conc} ({pct_missing_conc:.2f}%)

**Signal Statistics:**
- Mean UV (with pH): {mean_with_ph:.4f}
- Mean UV (without pH): {mean_without_ph:.4f if not np.isnan(mean_without_ph) else
    'N/A'}
- t-test Statistic: {t_stat:.4f}
- p-value: {p_value:.4f}

**Interpretation:**
- The t-test p-value of {p_value:.4f} indicates {'significant' if p_value < 0.05
    else 'not significant'} difference in signal intensity between runs with and
    without recorded pH.
"""

print(summary_text)
###

```

Listing 2: Code.2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant columns and drop NaNs in a single step using a boolean mask
mask = df[['UV_2_260_mAU', 'Conc_B_%']].notna().all(axis=1)

```

```

df_filtered = df[mask][['UV_2_260_mAU', 'Conc_B_%']]

# Debug print to check data state after filtering
print(f"Filtered data length: {len(df_filtered)}")
print(f"Filtered data shape: {df_filtered.shape}")
print(f"First few UV values: {df_filtered['UV_2_260_mAU'].head().tolist()}")

# Check if dataframe is empty after filtering
if df_filtered.empty:
    print("No valid data found after filtering.")
    summary_text = ""
## UV Signal Validity as Concentration Proxy Analysis Summary

### Correlation Analysis
- **Maximum Correlation Coefficient**: NaN
- **Optimal Lag**: NaN

### Recommendation
No data available for analysis.
"""
    print(summary_text)
    plt.close('all')
    exit()

# Handle single-run scenario
if len(df_filtered) == 1:
    print("Only one valid run found. Euclidean distance plot skipped.")
    # Calculate correlation for single run
    lags = np.arange(-5, 6)
    correlations = []
    for lag in lags:
        if lag < 0:
            corr = df_filtered['UV_2_260_mAU'].corr(df_filtered['Conc_B_%'].shift(-lag))
        else:
            corr = df_filtered['UV_2_260_mAU'].corr(df_filtered['Conc_B_%'].shift(lag))
        correlations.append(corr)

    correlations_df = pd.DataFrame({'Lag': lags, 'Correlation': correlations})

# Generate Heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(correlations_df.T, annot=True, cmap='coolwarm', center=0,
            xticklabels=['Lag'], yticklabels=[f'Lag_{i}' for i in lags])
plt.title('Correlation Coefficients: UV_2_260_mAU vs Conc_B_% across Lags')
plt.xlabel('Lag')
plt.ylabel('Correlation')
plt.tight_layout()
plt.savefig('/workspace/correlation_heatmap.png')
plt.close()

# Determine maximum correlation and optimal lag
max_corr = correlations_df['Correlation'].max()
if pd.isna(max_corr):
    max_corr = np.nan
    optimal_lag = np.nan
    recommendation = 'Indeterminate'
else:

```

```

        optimal_lag = correlations_df.loc[correlations_df['Correlation'].idxmax(),
        'Lag']
    if abs(max_corr) > 0.8:
        recommendation = 'Sufficient'
    else:
        recommendation = 'Insufficient'

    summary_text = f"""
## UV Signal Validity as Concentration Proxy Analysis Summary

### Correlation Analysis
- **Maximum Correlation Coefficient**: {max_corr:.4f}
- **Optimal Lag**: {optimal_lag}

### Recommendation
The correlation between UV_2_260_mAU and Conc_B_% is {recommendation}.
"""

    print(summary_text)
    plt.close('all')
else:
    # Calculate correlation between UV_2_260_mAU and Conc_B_% for lags -5 to +5
    using vectorized approach
    lags = np.arange(-5, 6)
    correlations = []
    for lag in lags:
        if lag < 0:
            corr = df_filtered['UV_2_260_mAU'].corr(df_filtered['Conc_B_%'].shift
            (-lag))
        else:
            corr = df_filtered['UV_2_260_mAU'].corr(df_filtered['Conc_B_%'].shift(
            lag))
        correlations.append(corr)

    correlations_df = pd.DataFrame({'Lag': lags, 'Correlation': correlations})

    # Generate Heatmap of correlation coefficients
    plt.figure(figsize=(10, 6))
    sns.heatmap(correlations_df.T, annot=True, cmap='coolwarm', center=0,
        xticklabels=['Lag'], yticklabels=[f'Lag_{i}' for i in lags])
    plt.title('Correlation Coefficients: UV_2_260_mAU vs Conc_B_% across Lags')
    plt.xlabel('Lag')
    plt.ylabel('Correlation')
    plt.tight_layout()
    plt.savefig('/workspace/correlation_heatmap.png')
    plt.close()

    # Determine maximum correlation and optimal lag dynamically
    max_corr = correlations_df['Correlation'].max()

    # Handle case where all correlations are NaN
    if pd.isna(max_corr):
        max_corr = np.nan
        optimal_lag = np.nan
        recommendation = 'Indeterminate'
    else:
        # Tie-breaking: prefer the largest lag if multiple lags share the maximum
        correlation
        optimal_lag = correlations_df.loc[correlations_df['Correlation'].idxmax(),
        'Lag']

```

```

    # Dynamic logic: Sufficient if |max_corr| > configurable baseline
    if abs(max_corr) > 0.8:
        recommendation = 'Sufficient'
    else:
        recommendation = 'Insufficient'

    # Output text summary in markdown
    summary_text = f"""
## UV Signal Validity as Concentration Proxy Analysis Summary

### Correlation Analysis
- **Maximum Correlation Coefficient**: {max_corr:.4f}
- **Optimal Lag**: {optimal_lag}

### Recommendation
The correlation between UV_2_260_mAU and Conc_B_% is {recommendation}.
"""

    print(summary_text)

```

Listing 3: Code_3